# AIStudio — Elements of an Architecture

*Version: 1.1.0*

*How AIStudio is put together, why it's built that way, and where it strains — written for someone who wants the "under the hood" picture without needing to read the code.*

---

## 1. The mental model

AIStudio is a **local, single-user retrieval-augmented question-answering application**. You give it a set of documents — a **corpus** — and you ask questions in plain language. It answers using *only* what's in those documents, and every claim carries a **citation** back to the exact source and page.

Nothing leaves the machine. The documents, the search index, and the language models all run locally. That one promise — *grounded answers with a path back to the source, on your own hardware* — is the product, and every architectural choice exists to keep it true.

Logically there are three layers and three always-on services.

**The three layers:**

1. **Ingest** (the write path) — a document is loaded, split into chunks, enriched, turned into numeric vectors, and stored. It runs once per document.
2. **Query** (the read path) — a question is turned into a vector, the most relevant chunks are retrieved and re-ranked, a prompt is assembled, and a local model writes the answer. It runs on every question.
3. **The interface** — a single offline page that talks to the backend over the local machine only.

**The three services:**

- A **vector store** that holds the searchable index — one collection per corpus.
- A **model host** that runs the embedding model (which turns text into vectors) and the answer-writing models. What that host actually is — and why it's a swappable component — is §7.

- A **backend** that orchestrates everything and exposes the application's interface and its local API.

---

## 2. The objects — what a corpus is made of

Everything hangs off a corpus. These are the pieces and how they relate.

- **The corpus** — a named document set. It maps one-to-one to a collection in the vector store and to a folder on disk holding its source files and its settings.
- **Source documents** — the files you actually search (annual filings, reports, your own documents). They are the ground truth; everything else is derived from them.
- **Chunks** — each source document is split into overlapping slices of roughly 1,200 characters. Each chunk is embedded and stored as one point in the vector store, alongside a small payload (which document and page it came from, and which firm it concerns). Chunks are what retrieval actually searches; the model never sees a whole document, only a handful of chunks.
- **The scope** — for a *downloaded* corpus, a list declaring which firms belong to it, keyed by a stable identifier (for financial filings, the LEI — the global legal-entity identifier). This is the membership roster; editing it is how you add or remove a firm.
- **Corpus settings** — per-corpus defaults (how many chunks to retrieve, the literal-vs-conceptual retrieval balance, the relevance floor, the answer model's creativity) plus a short description and routing guidance that is fed to the model on every question.
- **Knowledge bases** — optional, shared reference data: an *entity* knowledge base (canonical names, identifiers, and aliases for each firm) and a *glossary* (domain vocabulary, e.g. regulatory terms). They improve retrieval but are not required for it to work.

A useful way to hold all this: the **source documents and the scope are human-editable inputs**; the **chunks and the running index are derived outputs**, rebuilt from those inputs whenever you ingest.

---

## 3. The write path — how a document becomes searchable

When you add a document and ingest it:

1. **Load** — the file is read into plain text, with page boundaries preserved where the format has them.

2. **Chunk** — the text is split into ~1,200-character slices with ~200 characters of overlap, respecting sentence and paragraph boundaries so no idea is cut mid-thought. Chunk identifiers are deterministic, so re-ingesting a file overwrites its chunks in place rather than duplicating them.
3. **Enrich** (structured filings only) — for machine-readable filings, AIStudio reads the document's own embedded tags to learn *which firm* and *which fiscal year* it represents, then prepends a short label to every chunk — for example, `[Document: BlackRock, Inc. | BLK FY2025]`. This is one of the most important details in the system; §5 explains why.
4. **Embed** — each chunk's text is turned into a vector by the embedding model.
5. **Store** — the vectors and their payloads are written to the corpus's collection in the vector store.

Two properties worth internalizing:

- **Ingest is incremental.** Files that haven't changed are skipped. Adding one firm re-embeds one file and leaves the rest untouched; a forced run rebuilds the whole corpus from scratch.
- **Enrichment fires only on structured filings.** A tagged financial filing gets the firm label (and becomes precisely isolatable); a plain PDF you bring yourself does not, and is retrieved purely by similarity.

---

## 4. The read path — how a question becomes an answer

This is where retrieval and generation meet. On every question:

1. **(Optional) expand** the query with domain vocabulary, so jargon retrieves the right chunks.
2. **Embed** the question into a vector.
3. **Retrieve** from the vector store using a **hybrid** of two methods at once: dense vector search (matches *meaning*) and lexical/keyword search (matches *exact terms* — names, tickers, defined phrases). A single dial, the **Retrieval Mix**, sets the balance between them.
4. **Isolate** (when a firm is identified) — restrict retrieval to chunks that belong to that firm, so a question about one company can't be answered with another's text.
5. **Re-rank** — a small, fast model re-scores the top candidates against the exact question, floating the most relevant chunks to the top.
6. **Assemble the prompt** — the system instructions, the corpus's routing guidance, and the handful of retrieved chunks are combined.
7. **Generate** — the selected answer model writes a response *over the chunks it was handed*, and only those, marking each claim with a citation.

8. **Resolve citations** — the markers are mapped back to the source documents and pages, and the interface renders them as clickable references.

The crucial point: **the answer model is consulted exactly once, and it never sees the corpus** — only the small set of retrieved chunks. It composes and cites; it does not search. Retrieval quality, not model eloquence, is what makes an answer right.

---

## 5. What makes it more than "a vector database with a model attached"

Two mechanisms do most of the heavy lifting.

**The chunk label.** Prepending `[Document: <firm> | <ticker> FY<year>]` to every chunk does three jobs at once. It **binds the firm** to the chunk — financial filings say "the Firm" and "our" constantly, so without a re-anchor a model reading a mid-document chunk wouldn't know whose results it's looking at. It **binds the year**, which makes trend questions precise even when few chunks are retrieved. And it **widens recall**, because the firm's aliases and ticker now sit in the searchable text.

**Firm isolation.** Each chunk also carries, in its payload, the firm it concerns — recorded from the document's own declared identity, independent of the filename. A question about a named firm is restricted to that firm's chunks. This is the single most load-bearing idea in retrieval quality: on a large multi-firm corpus, *recognizing* a firm and adding its name to the query is **not** enough — without isolation, chunks from other firms that discuss the same topic densely will crowd out the right ones. The lever is exclusion, not emphasis.

---

## 6. The control surfaces

A handful of dials shape retrieval and generation:

| Dial | What it does |
|---|---|
| **Top K** | How many chunks are retrieved and handed to the model. Higher helps *breadth* (multi-firm questions); it doesn't sharpen focus. |
| **Retrieval Mix** | Literal ↔ conceptual. Left favors exact terms, tickers, defined phrases; right favors themes and meaning; the middle blends both. |

| Dial | What it does |
|---|---|
| **Score Threshold** | A relevance floor — chunks below it are dropped before the model sees them. Too high starves good chunks; lowering it helps when answers come back empty. |
| **Firm filter** | Restricts retrieval to one firm. |
| **Temperature** | The answer model's creativity. On document Q&A the retrieved context dominates, so low values are best and extremes mostly cost time. |

## 7. The inference layer — what the model host actually is

The **model host** named in §1 runs two kinds of model: the *embedding* model that turns text into vectors, and the *answer-writing* model that composes a response over the retrieved chunks. Both are large language models — and a model is not a program. It is a large array of trained numbers, its **weights**; on its own it does nothing. It needs a *runtime* to load those weights into memory and run the math that turns an input into an output. AIStudio uses **Ollama** as that runtime.

Ollama wraps the inference engine that does the actual tensor math (`llama.cpp`), runs it through Apple's **Metal** GPU backend on Apple Silicon, and adds the operational layer around it: a model registry, automatic download, memory management, and a local API. The weights ship **quantized** — compressed from the 16-bit training precision down to roughly 4-bit — so a 27-billion-parameter model occupies ~16–20 GB of unified memory instead of the ~54 GB it would need at full precision. Quantization is the reason a model this size runs on a laptop at all; the host then keeps a model resident for a short window after each call, so the next question skips the cold load.

The detail that matters architecturally: **because the runtime is separate from the weights, the inference layer is a swappable component.** AIStudio talks to the host over an **OpenAI-compatible API** — the de-facto interface that serving runtimes now converge on. The same calls would run, with a change of address and little more, against a high-throughput cloud serving stack the day the system had to answer many users at once rather than one — local today, scalable later, without rewriting the application above it. *(The concrete stack and pinned versions live in `dependencies.md`.)*

## 8. Why it's built this way — the design choices

- **Local-first.** Documents, index, and models all run on one machine. The reason is trust and privacy: a corpus of sensitive filings or internal documents never has to leave the building. It also makes the system fully inspectable.
- **Grounding over fluency.** The product is not "a chatbot that knows finance"; it is "answers you can check." That is why citations are first-class, why the model sees only retrieved evidence, and why the system is designed to *abstain* rather than guess when the evidence isn't there.
- **Hybrid retrieval as the default.** Pure conceptual search loses named entities — the shared concept dominates the vector and the firm's name barely registers. Blending in lexical matching restores the names. The balanced default is deliberate.
- **Isolation, not expansion.** Correctly answering a multi-firm question is a *filtering* problem, not a query-decoration problem (§5). The architecture treats firm recognition and firm isolation as separate steps and leans on the filter.
- **Enrichment is optional.** The knowledge bases improve recall and aliasing, but the core promise — isolate the right firm, cite the right source — works without them. They degrade gracefully.

---

## 9. How the system is benchmarked

Knowing *that* the system works is one thing; being able to **show how well, repeatably, and where it breaks** is another. AIStudio carries a benchmarking process, not just a benchmark number.

The process has four moves, run for each corpus:

1. **A fixed question set.** Each corpus has a versioned YAML of questions with expected-content keywords — the inputs are pinned so a run is reproducible, not ad-hoc.
2. **A mechanical pass.** The harness ( `ais_bench` ) runs every question, checks three conditions — keywords present, at least one citation, no hedging — and scores each 🟢 / 🟡 / 🔴 . This is cheap, automatic, and *deliberately naive*: it measures surface signals, not truth.
3. **A calibrated audit.** Because the mechanical score rewards confident-looking output, every run is then read against the **primary source** — the actual filing — and re-scored: correct-and-grounded, partial, miss, or grading-artifact. The gap between mechanical and calibrated is itself a finding [1].
4. **A pinned canonical suite.** A small set of runs — across both corpora, at the canonical depth — is fixed so the headline numbers can be regenerated on demand with `ais_bench`

`--canonical`, the same way the help corpus is rebuilt on each release. A companion verb, `ais_bench --batch`, runs the same sets against whatever models the *current* machine can actually hold — the pinned model is what makes numbers comparable, but it does not fit every Mac, and a suite nobody can run is not reproducible in any useful sense.

The design intent mirrors the rest of the architecture: **measure honestly, show the failures, make it reproducible.** The two frontiers the suite surfaces — multi-year/table synthesis on English text, and the language ceiling on non-English filings — are documented openly rather than hidden, because a system whose limits are known is more trustworthy than one whose limits are unstated [2]. The harness flags (`--corpus`, `--scope`, `--questions`, `--top-k`, `--canonical`, `--batch`) and how to read a report are covered in the harness manual [3]; the audited evidence and its synthesis are the canonical suite [4].

---

## The honest limits

Knowing where the trust promise strains is part of using AIStudio well:

- **Tables are hard.** [5] Dense numeric tables (an income statement, a multi-year capital table) retrieve poorly, because lexical scoring saturates across near-identical rows. A figure stated in prose is reliable; a precise number lifted from a grid should be treated as a claim to verify against the cited chunk, not a fact to repeat.
- **Citations vary by model.** Different answer models mark references differently, and the count can come back short or doubled. When precision matters, trust the *retrieved set* over the model's inline markers.
- **Language is a ceiling.** [6] Retrieval and extraction are strongest on English filings and fall off on other languages — a limit of the embedding and extraction layers, not of the isolation logic.
- **Your own documents aren't auto-isolated.** Plain PDFs you bring don't carry a firm label, so they're retrieved by similarity rather than isolated by firm. That's fine for single-topic document sets; it matters for multi-entity comparison.

The throughline: AIStudio's job is not to sound authoritative — it is to make its reasoning checkable. The architecture is the machinery that keeps every answer traceable back to a source you can open and read.

*** ***

## Additional Reading

The references below point into the rest of the AIStudio document corpus; bracket numbers appear in order of first mention above.

[1] **TUTORIAL.md** §5 — https://github.com/mbarberony/AIStudio/blob/main/TUTORIAL.md — how to read a benchmark: mechanical score vs. calibrated audit, and why the gap matters.

[2] **TUTORIAL.md** Annex 3 (the language ceiling) and Annex 4 (the table-cell frontier) — the two failure mechanisms, worked end to end.

[3] **BENCH_HARNESS.md** — https://github.com/mbarberony/AIStudio/blob/main/benchmarks/docs/BENCH_HARNESS.md — the harness manual: `ais_bench` flags, question-file format, reading a report, reproducing the suite with `--canonical`.

[4] **BENCH - Canonical Suite - README and Synthesis** — https://github.com/mbarberony/AIStudio/blob/main/benchmarks/docs/ — the audited evidence across all four canonical runs, with the cross-corpus synthesis.

[5] **TUTORIAL.md** Annex 4 — the multi-year, multi-column table case where a cell can be severed from the year that gives it meaning.

[6] **TUTORIAL.md** Annex 3 — why an English question against a non-English filing retrieves worse, and how the glossary does (and does not) help.

*** ***